

# Project Report: Memory Management Simulator

## 1. Introduction

Memory management is one of the core responsibilities of an operating system. It decides how memory is allocated, freed, reused, and tracked during program execution. This project implements a command-line memory management simulator in C++ to demonstrate important operating-system concepts such as allocation strategies, fragmentation, block coalescing, and cache hit/miss behavior.

The simulator does not allocate real process memory. Instead, it creates a controlled simulated memory space and allows the user to perform allocation, deallocation, memory dump, cache access, and statistics commands.

## 2. Objectives

The main objectives of this project are:

- To simulate contiguous memory allocation.
- To implement First Fit, Best Fit, and Worst Fit allocation strategies.
- To demonstrate block splitting during allocation.
- To demonstrate coalescing of adjacent free blocks during deallocation.
- To calculate memory utilization and external fragmentation.
- To simulate a two-level CPU cache hierarchy.
- To track cache hits, misses, and hit rate.
- To provide an interactive command-line interface for testing.

## 3. Technologies Used

| Technology        | Purpose                        |
|-------------------|--------------------------------|
| C++17             | Main programming language      |
| STL vector        | Simulated physical memory      |
| STL list          | Memory block metadata tracking |
| STL deque         | FIFO tracking in cache sets    |
| Makefile          | Build automation               |
| Bash test scripts | Command-based testing          |

## 4. Project Structure

```
memory-simulator/
├── docs/
│   ├── DesignDocument.md
│   └── ProjectReport.md
├── include/
│   ├── AllocatorStrategy.h
│   ├── BestFit.h
│   ├── Cache.h
│   ├── FirstFit.h
│   ├── MemoryManager.h
│   ├── SimulatorShell.h
│   └── WorstFit.h
├── src/
│   ├── main.cpp
│   ├── SimulatorShell.cpp
│   ├── allocator/
│   │   ├── BestFit.cpp
│   │   ├── FirstFit.cpp
│   │   └── MemoryManager.cpp
│   ├── WorstFit.cpp
│   ├── cache/
│   │   └── cache.cpp
│   └── tests/
│       ├── cache_test.txt
│       ├── fragmentation_test.txt
│       └── strategy_comparison_test.txt
├── Makefile
├── README.md
└── run_tests.sh
```

## 5. System Architecture

The simulator is divided into separate components to keep responsibilities clear.

| Component                   | Responsibility                                                         |
|-----------------------------|------------------------------------------------------------------------|
| main.cpp                    | Starts the simulator shell                                             |
| SimulatorShell              | Handles user commands and connects memory/cache components             |
| MemoryManager               | Manages simulated RAM, allocation, freeing, coalescing, and statistics |
| AllocatorStrategy           | Interface for allocation algorithms                                    |
| FirstFit, BestFit, WorstFit | Concrete allocation strategy implementations                           |
| Cache                       | Simulates set-associative cache behavior and statistics                |

## 6. Memory Management Design

The simulator represents physical memory as a `std::vector<uint8_t>`. This vector acts as the simulated RAM.

Memory block metadata is stored separately in a `std::list<Block>`. Each block contains:

- starting address
- block size
- free/used status

```
struct Block {
    size_t startAddress;
    size_t size;
    bool isFree;
};
```

The block list allows easy insertion of new free blocks during splitting and easy merging of adjacent free blocks during coalescing.

## 7. Allocation Process

When the user runs:

```
malloc 100
```

the simulator performs the following steps:

1. Checks whether an allocator strategy has been selected.
2. Uses the selected strategy to find a suitable free block.
3. If no block is available, allocation fails.
4. If the selected block is larger than requested, the block is split.
5. The requested part is marked as used.
6. The remaining part stays free.
7. The starting address of the allocated block is returned.

## 8. Deallocation Process

When the user runs:

```
free 100
```

the simulator:

1. Searches for a block whose starting address is 100.
2. Checks whether the address is valid.
3. Checks whether the block is already free.
4. Marks the block as free.
5. Calls the coalescing function.

Coalescing merges adjacent free blocks to reduce external fragmentation.

## 9. Allocation Strategies

## 9.1 First Fit

First Fit scans the memory block list from the beginning and selects the first free block large enough for the request.

Advantages:

- Simple to implement.
- Often fast in practice.

Disadvantages:

- Can create many small fragments near the beginning of memory.

## 9.2 Best Fit

Best Fit scans the entire block list and chooses the free block that leaves the smallest leftover space.

Advantages:

- Tries to use memory space efficiently.
- Reduces immediate waste in the selected block.

Disadvantages:

- Scans the full list.
- Can create very small unusable fragments.

## 9.3 Worst Fit

Worst Fit scans the entire block list and chooses the largest available free block.

Advantages:

- Leaves behind relatively larger free blocks.

Disadvantages:

- Breaks up the largest contiguous memory areas quickly.

# 10. Fragmentation Analysis

The simulator calculates:

- total memory
- used memory
- free memory
- utilization percentage

- external fragmentation percentage
- largest free block

External fragmentation is calculated using:

$$\text{External Fragmentation} = 1 - (\text{Largest Free Block} / \text{Total Free Memory})$$

If all free memory is present in one large block, fragmentation is low. If free memory is split across many small blocks, fragmentation is high.

## 11. Cache Simulation

The project also simulates a two-level CPU cache hierarchy.

| Cache | Size       | Associativity | Block Size | Replacement Policy |
|-------|------------|---------------|------------|--------------------|
| L1    | 1024 bytes | 2-way         | 64 bytes   | FIFO               |
| L2    | 4096 bytes | 4-way         | 64 bytes   | FIFO               |

When the user runs:

```
read 64
```

the simulator checks:

1. L1 cache
2. L2 cache
3. Physical memory

If the address is found in L1, it is an L1 hit. If not, the simulator checks L2. If it is missing from both caches, the simulator reports a physical memory access.

## 12. FIFO Replacement

Each cache set stores cache lines and an insertion-order queue. When a set has empty space, the new cache block is inserted into an empty line. When the set is full, the oldest inserted line is replaced.

This models FIFO replacement:

```
First block inserted = first block removed
```

## 13. Command-Line Interface

The simulator provides an interactive shell.

| Command                  | Description                  |
|--------------------------|------------------------------|
| init memory <size>       | Initializes simulated memory |
| set allocator <strategy> | Sets allocation strategy     |
| malloc <size>            | Allocates memory             |

|                  |                                       |
|------------------|---------------------------------------|
| alloc <size>     | Alias for malloc                      |
| free <address>   | Frees memory block                    |
| dump memory      | Prints memory layout                  |
| read <address>   | Simulates memory read                 |
| write <address>  | Simulates memory write                |
| access <address> | Generic cache access command          |
| reset cache      | Clears cache contents and cache stats |
| stats            | Prints memory and cache statistics    |
| help             | Shows command list                    |
| exit             | Exits the simulator                   |

## 14. Example Execution

Input:

```
init memory 1024
set allocator best fit
malloc 100
malloc 200
malloc 50
free 100
malloc 150
dump memory
read 64
read 65
stats
exit
```

Explanation:

- 1024 bytes of memory are initialized.
- Best Fit allocator is selected.
- Three blocks are allocated.
- One block is freed.
- A new 150-byte block is allocated into the best matching free space.
- The memory layout is displayed.
- Two memory accesses are simulated through cache.
- Final memory and cache statistics are printed.

## 15. Test Cases

### 15.1 Fragmentation Test

File:

```
tests/fragmentation_test.txt
```

Purpose:

- allocates multiple blocks
- frees selected blocks
- shows memory fragmentation
- prints memory statistics

## 15.2 Cache Test

File:

```
tests/cache_test.txt
```

Purpose:

- demonstrates cache misses and hits
- shows spatial locality
- verifies cache hit-rate calculation

## 15.3 Strategy Comparison Test

File:

```
tests/strategy_comparison_test.txt
```

Purpose:

- compares Best Fit and Worst Fit
- shows that each strategy can choose a different block for the same allocation request

## 16. Build and Run

Build with Make:

```
make
```

Run:

```
./bin/memsim
```

If using the .exe version:

```
./bin/memsim.exe
```

Run tests:

```
./run_tests.sh
```

## 17. Limitations

- The simulator does not implement virtual memory.
- It does not implement paging or page replacement algorithms.
- It does not implement a Buddy Memory Allocation System.
- It does not store actual data values in allocated memory blocks.
- Cache lines store tag and valid-bit information only.
- It assumes a single-process simulation environment.

## 18. Conclusion

This project demonstrates important operating-system memory concepts through a simple command-line simulator. It shows how different allocation strategies affect memory layout, how freeing memory can create fragmentation, how coalescing reduces fragmentation, and how cache locality affects memory access performance.

The simulator provides a practical way to understand memory allocation and cache behavior without requiring kernel-level programming.